

MODEL PREDICTIVE CONTROL

Lecture Handout

April 16, 2026

Contents

I	Introduction	5
1	Introduction to Model Predictive Control	7
1.1	Optimization-Based Control	7
1.2	Concept of Model Predictive Control	8
1.2.1	Why constraints matter	9
1.3	Applications of MPC	9
1.4	A Brief History of MPC	10
1.5	Summary and Main Aspects	11
2	Systems theory basics	13
2.1	Models of Dynamic Systems	13
2.1.1	Nonlinear continuous-time state-space models	13
2.1.2	Linear time-invariant continuous-time models	14
2.1.3	Linearization around an operating point	14
2.1.4	Discrete-time state-space models	16
2.1.5	Euler discretization	16
2.1.6	Exact discretization of linear systems	17
2.1.7	Solution of linear discrete-time systems	17
2.2	Analysis of LTI Discrete-Time Systems	17
2.2.1	Coordinate transformations	18
2.2.2	Stability of linear discrete-time systems	18
2.2.3	Controllability and stabilizability	18
2.2.4	Observability and detectability	19
2.3	Analysis of Nonlinear Discrete-Time Systems	20
2.3.1	Lyapunov stability of nonlinear systems	20
2.3.2	Lyapunov functions	20
2.3.3	Indirect Lyapunov method	21
2.3.4	Lyapunov stability of linear systems	21
2.4	Summary	22
3	Unconstrained Control	23
3.1	Introduction to Linear Quadratic Optimal Control	23
3.1.1	Finite-horizon linear quadratic regulation	23
3.2	Batch Approach	24
3.3	Recursive Approach and Dynamic Programming	25
3.3.1	Cost-to-go and one-step optimization	25
3.3.2	Bellman's principle of optimality	26
3.3.3	Comparison of batch and recursive viewpoints	27
3.4	Receding Horizon Interpretation	27
3.5	Infinite-Horizon LQR	28
3.5.1	Algebraic Riccati equation	28

3.5.2	Stability of infinite-horizon LQR	28
3.6	Terminal Weight and Quasi-Infinite-Horizon Thinking	30
3.7	Summary	31
4	Introduction to ConvexOptimization	33
II	Nominal MPC	35
5	Constrained Finite Time Optimal Control	37
6	Invariance	39
7	Feasibility and Stability	41
III	Practical MPC	43
8	Practical Issues in MPC	45
IV	Robust MPC	47
9	Robust MPC 1	49
10	Robust MPC 2	51
V	Implementation Methods	53
11	Implementation Methods	55

Part I

Introduction

Chapter 1

Introduction to Model Predictive Control

1.1 Optimization-Based Control

A central idea behind model predictive control is that the control action is not generated by a fixed pre-computed feedback law alone, but by repeatedly solving an optimization problem online. This point of view is best understood by comparing it with the classical control loop. In a standard feedback architecture, the controller receives a reference and the measured output of the plant, and then applies an input according to a control law that has been designed in advance. In optimization-based control, this fixed controller block is replaced, at least conceptually, by an optimizer.

The optimizer uses a model of the plant to predict how the system will evolve under candidate future inputs. Based on these predictions, it computes an input sequence that best satisfies the desired control objectives while respecting the relevant constraints. The first important shift is therefore methodological: instead of asking directly for a controller formula, we formulate a decision problem whose solution determines the input to be applied.

This approach is particularly natural when the control objective is tightly coupled with physical limitations and safety requirements. Consider, for instance, the problem of driving a race car around a track. One may wish to minimize the lap time, but this objective cannot be pursued arbitrarily. The car must stay on the road, avoid collisions, respect limitations on tire friction, and remain within admissible acceleration and steering capabilities. The control task is therefore inherently constrained. A meaningful strategy is to look ahead, predict the upcoming road geometry and the effect of future decisions, and then plan a feasible trajectory that achieves the best possible performance. This is precisely the kind of reasoning that optimization-based control formalizes.

The same idea can be described algorithmically as follows. At a given time instant, one solves an optimization problem to compute a planned trajectory and a corresponding sequence of future control actions. From this sequence, only the first control move is actually implemented. At the next sampling instant, the state is measured again, the prediction is updated, and a new optimization problem is solved. In this way, planning and feedback are combined in a single framework.

Optimization-based control

The key idea is to compute the control input by solving, at each sampling instant, a finite-dimensional optimization problem built from three ingredients: a performance objective, a

predictive model of the plant, and a set of constraints.

1.2 Concept of Model Predictive Control

Model predictive control, usually abbreviated as MPC, is the most prominent realization of optimization-based control. The name itself summarizes its defining features. It is *model-based* because future behavior is predicted through an internal model of the plant. It is *predictive* because the controller explicitly reasons over a future horizon. Finally, it is a *control* method because the resulting optimization problem is solved repeatedly in closed loop in order to decide the plant input.

A useful way to understand MPC is to compare it with more classical control design philosophies. In classical control, one typically designs a controller C for a plant P so as to achieve good disturbance rejection, limited sensitivity to measurement noise, and robustness to modeling uncertainty. The dominant language is often that of transfer functions, loop shaping, and frequency-domain specifications. By contrast, MPC is naturally expressed in the **time domain**. Its strength lies in the **explicit treatment of constraints**, both on the manipulated inputs and on the system variables. This makes it especially appealing in applications where safety, actuator saturation, and operating envelopes are of central importance.

Indeed, all physical systems are constrained. Inputs cannot be arbitrarily large because actuators have limits. Outputs and states are often restricted by performance or safety considerations, such as maximum temperature, pressure, velocity, or overshoot. In many practical settings, the economically or operationally optimal regime lies close to such limits. Traditional control methods often handle constraints only indirectly, for example by tuning the set-point conservatively or by adding ad hoc supervisory logic. MPC instead incorporates the constraints directly into the control design. This allows operation closer to the admissible boundaries, while still maintaining systematic decision-making.

Mathematically, MPC can be formulated over a finite prediction horizon of length N . At a generic time instant k , one considers the current state $x(k)$ and searches for a sequence of future control inputs

$$U_k = \{u_k, u_{k+1}, \dots, u_{k+N-1}\}$$

that minimizes a cumulative performance criterion. A standard formulation is

$$U_k^*(x(k)) := \arg \min_{U_k} \sum_{i=0}^{N-1} \ell(x_{k+i}, u_{k+i})$$

subject to the system dynamics

$$x_{k+i+1} = Ax_{k+i} + Bu_{k+i},$$

the initial condition

$$x_k = x(k),$$

and the constraints

$$x_{k+i} \in \mathcal{X}, \quad u_{k+i} \in \mathcal{U},$$

for all relevant indices i .

This compact expression already contains the essential ingredients of MPC. The objective function defines what the controller tries to optimize. The model provides predictions of future behavior. The constraint sets $\mathcal{X}$ and $\mathcal{U}$ encode admissible operating conditions. The decision variables are the future inputs across the prediction horizon.

The online operation of MPC follows a receding-horizon principle. At each sampling time, the current state is measured or estimated, the finite-horizon optimization problem is solved, and an optimal input sequence

$$U_k^* = \{u_k^*, u_{k+1}^*, \dots, u_{k+N-1}^*\}$$

is obtained. However, only the first element u_k^* is applied to the plant. After one sampling interval, the state is updated and the entire optimization is solved again from the new initial condition. The horizon therefore moves forward in time, which is why one also speaks of a *receding horizon* strategy.

This repeated re-optimization is what **introduces feedback**. If the plant is disturbed, if the model is imperfect, or if the measured state differs from the previously predicted one, the next optimization step automatically adjusts the future plan. The controller does not blindly execute an open-loop plan; instead, it continuously replans using fresh information.

Receding horizon principle

MPC computes an *optimal sequence* of future control moves, but implements only the *first* one. Feedback arises because the optimization problem is solved again at the next sampling instant using the newly measured or estimated state.

1.2.1 Why constraints matter

The explicit inclusion of constraints is one of the main reasons for the success of MPC. Suppose one wants to regulate a process output to a desired set-point while an upper safety limit must never be violated. A classical controller may achieve fast convergence but produce overshoot that crosses the admissible bound. To avoid this, one may reduce aggressiveness or move the set-point away from the constraint, which typically leads to suboptimal operation. MPC offers a more systematic alternative: the constraint is included directly in the optimization problem, and the computed control action is chosen with full awareness of that limit.

This ability to reason ahead is crucial. The optimizer can anticipate that a constraint will become active in the near future and can begin adjusting the input before the violation occurs. In this sense, MPC is predictive not only in the use of a model, but also in the way it handles constraints proactively rather than reactively.

1.3 Applications of MPC

One of the remarkable features of MPC is the **breadth of its application domain**. The same conceptual framework appears in problems evolving at extremely different time scales.

At very fast time scales, optimization-based control ideas arise in computer and electronic systems, where decisions may need to be made on the **nanosecond or microsecond** level. Slightly slower, but still extremely fast, are power system applications, where control actions and operational decisions can be taken in the microsecond range depending on the problem formulation and hardware layer under consideration.

At the **millisecond** scale, traction control and automotive systems provide a classical application area. Here, one must coordinate performance objectives with safety and actuator limits in real time. On the scale of **seconds**, building control becomes relevant: thermal dynamics are slow enough to allow relatively long prediction horizons, yet constraints and energy-efficiency objectives play a major role. At the scale of **minutes and hours**, refinery operation and process control have historically been among the main industrial drivers for MPC. At even slower scales, **days or weeks**, one finds planning and scheduling problems such as train scheduling, nurse

rostering, and production planning, where the same optimization-and-prediction philosophy remains useful.

These examples show that MPC should not be viewed only as a single algorithm for fast dynamical systems. Rather, it is a general methodology for repeated decision-making under dynamical models and constraints. The exact mathematical models, optimization problems, and computational requirements vary greatly across applications, but the underlying structure remains the same.

1.4 A Brief History of MPC

The historical roots of MPC go back to the early idea of using optimization methods directly inside control algorithms. An early milestone is the work of A. I. Propoi in 1963, where linear programming ideas were proposed for sampled-data automatic systems. This already contains the seed of a now-familiar idea: control can be posed as the repeated solution of an optimization problem.

The modern industrial development of MPC is commonly traced to the 1970s. A key step was the work of Richalet and co-authors in 1978 on what became known as IDCOM, short for *Identification and Command*. Their approach used an impulse response model of the plant, considered a finite prediction horizon, and formulated a quadratic performance objective. Future plant behavior was described relative to a desired reference trajectory, and practical input/output constraints were included in an ad hoc manner. Because the resulting controller was not expressed as a conventional transfer function but rather as the outcome of an iterative computational procedure, the approach was described as heuristic predictive control. At this stage, the framework was mainly aimed at stable plants and was strongly motivated by industrial needs.

A closely related and highly influential development came from Cutler and collaborators. In the late 1970s, Cutler proposed ideas that were later implemented at Shell under the name *Dynamic Matrix Control* (DMC). The 1979 work by Cutler and Ramaker is particularly important in the industrial history of MPC. DMC used a step response model of the plant, a finite-horizon quadratic objective, and a prediction mechanism that aimed at tracking the set-points as closely as possible. The optimal manipulated variables were computed through a least-squares problem, and additional equations were introduced online to account for constraints. The name “dynamic matrix” refers precisely to the structured matrix that arises in this least-squares formulation. A further important step occurred in 1983, when Cutler, Morshedi, and Haydel emphasized the formulation of the controller as a standard quadratic program. This was conceptually decisive, because it provided a systematic mathematical framework for handling constraints rather than relying only on heuristic modifications. From that point on, MPC gradually evolved from an industrial engineering practice into a discipline with a firm optimization-theoretic foundation.

In the mid 1990s, much theoretical work was devoted to understanding under which conditions one can guarantee recursive feasibility and closed-loop stability. This period established many of the fundamental results that are now part of the standard theory of nominal MPC. During the 2000s, research expanded toward tractable robust MPC methods, nonlinear MPC, hybrid systems, and fast MPC algorithms tailored to increasingly demanding real-time requirements. In the 2010s, important developments included stochastic MPC, distributed and large-scale MPC, and economic MPC, where the objective is not merely set-point tracking but direct optimization of process performance. In the 2020s, a major direction has been the interaction between MPC and learning, leading to a growing body of work on learning-based MPC.

1.5 Summary and Main Aspects

Model predictive control can therefore be summarized as a control methodology in which, at every sampling instant, one measures or estimates the current state, predicts future behavior with an internal model, solves a constrained finite-horizon optimization problem, and applies only the first input of the optimal sequence. The procedure is then repeated at the next sampling instant.

Its main appeal lies in the fact that performance objectives and operating constraints are treated within a single optimization-based framework. This gives the designer a systematic way to handle actuator limits, safety requirements, and multi-variable interactions while maintaining a feedback structure through repeated re-planning.

At the same time, MPC also comes with **important challenges**. First, the optimization problem must be solved reliably and fast enough for real-time use. Second, the finite-horizon problem may become infeasible if the constraints are too restrictive or if the system is driven into an unfavorable region. Third, closed-loop stability is not automatic and must typically be enforced or analyzed through suitable design choices. Finally, the closed-loop behavior may be sensitive to model mismatch and disturbances, which motivates the study of robustness and the many extensions of the nominal MPC framework.

Chapter 2

Systems theory basics

2.1 Models of Dynamic Systems

Model predictive control relies on a mathematical model of the system to predict future behavior. This is one of the fundamental ingredients of the entire framework: without a model, there is no basis for forecasting the effect of future control inputs and therefore no basis for optimization-based planning. In practice, one also needs a state estimator, a suitable optimal control problem formulation, a numerical optimization procedure, and finally a verification that the resulting closed-loop system behaves as desired. In this sense, system theory provides the language and tools on which MPC is built.

Dynamic-system models can be classified in several ways. One may distinguish between state-space and transfer-function descriptions, between linear and nonlinear models, between time-varying and time-invariant dynamics, between continuous-time and discrete-time formulations, and between deterministic and stochastic models. In this handout, as in the lecture, the focus is mainly on deterministic models. Physical systems derived from first principles are typically described by nonlinear, time-invariant, continuous-time state-space models, whereas the standard models used in MPC are most often linear, time-invariant, discrete-time state-space models. A substantial part of the modeling workflow therefore consists in transforming the former into the latter in a meaningful way.

2.1.1 Nonlinear continuous-time state-space models

A very general class of dynamical systems can be written in **continuous-time state-space form** as

$$\dot{x} = g(x, u), \quad y = h(x, u),$$

where $x \in \mathbb{R}^n$ is the state vector, $u \in \mathbb{R}^m$ is the input, and $y \in \mathbb{R}^p$ is the output. The map $g : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ describes the system dynamics, while $h : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^p$ specifies which quantities are measured or observed. This representation is extremely flexible and can describe a very broad range of physical systems.

An important reason why the state-space formalism is so useful is that higher-order ordinary differential equations can always be rewritten as systems of first-order equations. If one starts from an n th-order differential equation

$$x^{(n)} + g_n(x, \dot{x}, \ddot{x}, \dots, x^{(n-1)}) = 0$$

suitable state variables can be introduced by stacking the variable and its derivatives up to order $n - 1$. This is possible by defining

$$x_{j+1} = x^{(j)}, \quad j = 0, 1, \dots, n - 1,$$

where $x^{(0)}$ is simply x . The original equation can then be rewritten as

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_{n-1} \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} x_2 \\ x_3 \\ \vdots \\ x_n \\ -g_n(x_1, x_2, \dots, x_n) \end{bmatrix}.$$

The original equation is then transformed into n coupled first-order equations. This conversion is not merely formal: it provides a standard representation in which analysis and controller design can be carried out systematically.

The **drawback** of such nonlinear descriptions is that, although they are general and physically meaningful, their analysis and controller synthesis are often difficult. For this reason, one often replaces them locally by linear approximations around a chosen operating point.

2.1.2 Linear time-invariant continuous-time models

A continuous-time linear time-invariant system is written as

$$\dot{x} = A_c x + B_c u, \quad y = Cx + Du,$$

where A_c , B_c , C , and D are constant matrices of compatible dimensions. Compared with the nonlinear case, this model class is much more **restrictive**, but it has a decisive **advantage**: a vast mathematical theory is available for its analysis and control design.

One important feature of linear systems is that their solution can be written explicitly. If

$$\dot{x}(t) = A_c x(t) + B_c u(t), \quad x(t_0) = x_0,$$

then the state at time t is given by

$$x(t) = e^{A_c(t-t_0)} x_0 + \int_{t_0}^t e^{A_c(t-\tau)} B_c u(\tau) d\tau,$$

where the matrix exponential is defined by

$$e^{A_c t} := \sum_{n=0}^{\infty} \frac{(A_c t)^n}{n!}.$$

This expression makes the effect of the initial condition and the input signal fully explicit.

2.1.3 Linearization around an operating point

Most real systems are nonlinear, yet linear models are much easier to analyze and use in controller design. The standard compromise is therefore to **linearize the nonlinear dynamics around an operating point**. The idea is that if the system is to operate near a desired equilibrium, then a first-order approximation may be sufficiently accurate in a neighborhood of that point.

The mathematical basis is the first-order Taylor expansion. If a nonlinear function is expanded around a point $\bar{x}$, then near $\bar{x}$ it can be approximated by its value at $\bar{x}$ plus the Jacobian times the deviation from that point. The first order Taylor approximation of a function $f(\cdot)$ around $\bar{x}$ is

$$f(x) \approx f(\bar{x}) + \left. \frac{\partial f}{\partial x^\top} \right|_{x=\bar{x}} (x - \bar{x}) \quad \text{with} \quad \frac{\partial f}{\partial x^\top} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \dots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

Applied to a nonlinear state-space system

$$\dot{x} = g(x, u), \quad y = h(x, u),$$

and linearized around a stationary operating point (x_s, u_s) satisfying

$$g(x_s, u_s) = 0,$$

one obtains a linear model in deviation variables:

$$\dot{x} = \underbrace{g(x_s, u_s)}_{=0} + \underbrace{\frac{\partial g}{\partial x^\top} \Big|_{x=x_s, u=u_s}}_{=A_c} \underbrace{(x - x_s)}_{\Delta x} + \underbrace{\frac{\partial g}{\partial u^\top} \Big|_{x=x_s, u=u_s}}_{=B_c} \underbrace{(u - u_s)}_{\Delta u}$$

Thus

$$\dot{x} - \underbrace{\dot{x}_s}_{=0} = \Delta \dot{x} = A_c \Delta x + B_c \Delta u$$

Proceeding in the same way for the output equation, one obtains

$$\Delta y = \underbrace{\frac{\partial h}{\partial x^\top} \Big|_{x=x_s, u=u_s}}_{=C} \underbrace{(x - x_s)}_{\Delta x} + \underbrace{\frac{\partial h}{\partial u^\top} \Big|_{x=x_s, u=u_s}}_{=D} \underbrace{(u - u_s)}_{\Delta u}$$

Thus we obtain the linearized model

$$\Delta \dot{x} = A_c \Delta x + B_c \Delta u, \quad \Delta y = C \Delta x + D \Delta u,$$

where

$$A_c = \frac{\partial g}{\partial x^\top} \Big|_{x=x_s, u=u_s}, \quad B_c = \frac{\partial g}{\partial u^\top} \Big|_{x=x_s, u=u_s},$$

$$C = \frac{\partial h}{\partial x^\top} \Big|_{x=x_s, u=u_s}, \quad D = \frac{\partial h}{\partial u^\top} \Big|_{x=x_s, u=u_s}.$$

After linearization, it is common to drop the Δ notation and simply denote deviations again by x , u , and y .

The pendulum example clearly illustrates both the usefulness and the limitations of this procedure. Around a chosen equilibrium, such as a constant nonzero angle with zero angular velocity, the nonlinear sine term is replaced by its local linear approximation. The resulting matrices depend on the selected operating point. The linearized dynamics provide a good approximation only in a small neighborhood: for small angles and velocities the behavior of the linear and nonlinear systems is close, whereas farther away the approximation deteriorates. This local nature of linearization is essential to keep in mind in MPC modeling.

Linearization

Linearization replaces a nonlinear model by a local linear approximation around a stationary operating point. It is extremely useful for analysis and design, but its validity is inherently local: it can only be trusted in a sufficiently small neighborhood of the linearization point.

2.1.4 Discrete-time state-space models

Since controllers are typically implemented on digital hardware, MPC is naturally formulated in discrete time. A **discrete-time system** is described by difference equations of the form

$$x(k+1) = g(x(k), u(k)), \quad y(k) = h(x(k), u(k)),$$

where the input and output are defined only at discrete instants indexed by $k \in \mathbb{Z}_+$. Some systems are inherently discrete, such as an account balance updated every month, but most control systems originate in continuous time and are then discretized for implementation. The sampling instants are typically of the form

$$t_{k+1} = t_k + T_s,$$

where T_s is the sampling time. A standard assumption is that the control input is held constant between two consecutive sampling instants,

$$u(t) = u(t_k), \quad t \in [t_k, t_{k+1}),$$

which is the usual zero-order-hold implementation.

2.1.5 Euler discretization

For nonlinear systems, one simple discretization method is the forward Euler approximation. Starting from the continuous-time model

$$\dot{x}_c(t) = g_c(x_c(t), u_c(t)), \quad y_c(t) = h_c(x_c(t), u_c(t)),$$

one approximates the derivative by

$$\dot{x}_c(t) \approx \frac{x_c(t + T_s) - x_c(t)}{T_s}.$$

where T_s is the sampling time. Introducing the notation

$$x(k) = x_c(t_0 + kT_s), \quad u(k) = u_c(t_0 + kT_s),$$

one obtains the discrete-time approximation

$$x(k+1) = x(k) + T_s g_c(x(k), u(k)), \quad y(k) = h_c(x(k), u(k)).$$

Under suitable regularity assumptions and for sufficiently small sampling times, the discrete-time trajectory approximates the continuous-time one well.

For linear continuous-time systems,

$$\dot{x}(t) = A_c x(t) + B_c u(t), \quad y(t) = C_c x(t) + D_c u(t),$$

Euler discretization yields

$$x(k+1) = A x(k) + B u(k), \quad y(k) = C x(k) + D u(k),$$

with

$$A = I + T_s A_c, \quad B = T_s B_c, \quad C = C_c, \quad D = D_c.$$

This is straightforward and often useful, although it is not exact.

2.1.6 Exact discretization of linear systems

For linear systems under zero-order hold, one can derive an exact discrete-time model. Using the explicit continuous-time solution and assuming that the input remains constant over one sampling interval, one obtains

$$x(t_{k+1}) = e^{A_c T_s} x(t_k) + \left(\int_0^{T_s} e^{A_c(T_s-\tau)} B_c d\tau \right) u(t_k).$$

Hence the exact discrete-time system is

$$x(k+1) = Ax(k) + Bu(k),$$

with

$$A = e^{A_c T_s}, \quad B = \int_0^{T_s} e^{A_c(T_s-\tau)} B_c d\tau.$$

If A_c is invertible, then B can also be written as

$$B = A_c^{-1}(A - I)B_c.$$

This model is **exact** in the sense that it predicts the state of the continuous-time system exactly at the sampling instants, provided the zero-order-hold assumption is satisfied.

2.1.7 Solution of linear discrete-time systems

One major advantage of discrete-time linear systems is that their future state can be written explicitly as a function of the initial condition and the input sequence. Starting from

$$x(k+1) = Ax(k) + Bu(k),$$

successive substitution gives

$$x(k+N) = A^N x(k) + \sum_{i=0}^{N-1} A^i B u(k+N-1-i).$$

Thus, for fixed matrices A and B , the predicted state over a horizon is a **linear function of the current state and of the future inputs**. This observation is fundamental in MPC, because it allows the finite-horizon optimization problem to be written directly in terms of the decision variables and the known initial state.

The modeling discussion can be summarized as follows. Continuous-time nonlinear models are general and physically natural, but they are difficult to analyze and use directly in predictive control. Linearization gives a local continuous-time approximation around an operating point. Discretization then converts the model into a form that is directly compatible with digital control implementation. For linear systems, exact zero-order-hold discretization is available, while for general nonlinear systems one often relies on approximate methods such as Euler discretization, whose quality depends strongly on the sampling time. These are precisely the model transformations that make standard MPC formulations possible.

2.2 Analysis of LTI Discrete-Time Systems

Once a model has been put into discrete-time LTI form, one can study several structural properties that are central in estimation and control. Among the most important are the behavior of the system under coordinate changes, asymptotic stability, controllability, and observability. These notions form the classical backbone of linear systems theory and play a direct role in MPC.

2.2.1 Coordinate transformations

Consider the discrete-time LTI system

$$x(k+1) = Ax(k) + Bu(k), \quad y(k) = Cx(k) + Du(k).$$

The input-output behavior is completely determined by the initial state and the input sequence. However, the choice of state variables is **not unique**. Different state representations may generate exactly the same input-output behavior, and some representations may be more convenient than others for analysis or design.

If one introduces a new state variable

$$\tilde{x}(k) = Tx(k),$$

where T is invertible, then the transformed system becomes

$$\tilde{x}(k+1) = \tilde{A}\tilde{x}(k) + \tilde{B}u(k), \quad y(k) = \tilde{C}\tilde{x}(k) + \tilde{D}u(k),$$

with

$$\tilde{A} = TAT^{-1}, \quad \tilde{B} = TB, \quad \tilde{C} = CT^{-1}, \quad \tilde{D} = D.$$

The input and output remain unchanged; only the internal coordinates have changed. Such transformations are useful because they can simplify the structure of the matrices and thereby reveal important properties more transparently.

2.2.2 Stability of linear discrete-time systems

For the autonomous system

$$x(k+1) = Ax(k),$$

asymptotic stability means that every trajectory converges to the origin as $k \rightarrow \infty$. A fundamental theorem states that this happens if and only if all eigenvalues of A lie strictly inside the unit circle, that is,

$$|\lambda_j| < 1, \quad j = 1, \dots, n.$$

This is the discrete-time analogue of the continuous-time condition $\text{Re}(\lambda_j) < 0$.

The intuition becomes especially clear when the system can be diagonalized. In suitable coordinates, the system matrix becomes diagonal, and each transformed state component evolves independently as multiplication by the corresponding eigenvalue. If the magnitude of every eigenvalue is less than one, repeated multiplication drives each component to zero. If any eigenvalue has magnitude greater than or equal to one, some component fails to decay. Even when diagonalization is not possible and Jordan forms must be used, the same conclusion remains valid.

2.2.3 Controllability and stabilizability

A system

$$x(k+1) = Ax(k) + Bu(k)$$

is **controllable** if, for any initial state $x(0)$ and any desired target state x^* , there exists a finite time N and an input sequence $\{u(0), u(1), \dots, u(N-1)\}$ that steers the system exactly to that target, i.e. $x(N) = x^*$. In discrete time, this notion is often also called reachability. By expanding the state recursively:

$$x^* = x(N) = A^N x(0) + \sum_{i=0}^{N-1} A^i B u(N-1-i) = A^N x(0) + \begin{bmatrix} B & AB & \cdots & A^{N-1}B \end{bmatrix} \begin{bmatrix} u(N-1) \\ u(N-2) \\ \vdots \\ u(0) \end{bmatrix}$$

It follows from the **Carley-Hamilton** theorem that A^k can be expressed as a linear combination of $I, A, \dots, A^{n-1}$, so that for all $N \geq n$

$$\text{range} \begin{bmatrix} B & AB & \dots & A^{N-1}B \end{bmatrix} = \begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix}$$

We therefore conclude that the reachable states are precisely those that can be generated by the columns of the **controllability matrix**

$$\mathcal{C} = \begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix},$$

this means that the reachable states are precisely those in the column space of $\mathcal{C}$. The system is controllable **if and only if**

$$\text{rank}(\mathcal{C}) = n.$$

A weaker notion is **stabilizability**. A system is stabilizable if the unstable part of the dynamics can still be influenced by the input, even if some already stable modes are uncontrollable. More precisely, the system is stabilizable if all uncontrollable modes are stable. Controllability always implies stabilizability, but the converse need not hold. In many feedback-design settings, stabilizability is the essential requirement.

Stabilizability can be checked using the following condition

$$\text{if } \text{rank} \left(\begin{bmatrix} \lambda_j I - A & B \end{bmatrix} \right) = n \quad \forall \lambda_j \in \Lambda_A^+ \Rightarrow (A, B) \text{ is stabilizable}$$

where Λ_A^+ is the set of eigenvalues of A lying on or outside the unit circle.

2.2.4 Observability and detectability

Dual to controllability is the notion of **observability**. Consider the zero-input system

$$x(k+1) = Ax(k), \quad y(k) = Cx(k).$$

The system is observable if the initial state can be uniquely reconstructed from a finite sequence of output measurements. By stacking the outputs over time, one obtains the linear relation

$$\begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(n-1) \end{bmatrix} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} x(0).$$

The matrix

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

is called the **observability matrix**, and the system is observable **if and only if**

$$\text{rank}(\mathcal{O}) = n.$$

Again, the use of only the first n blocks is justified by the Cayley-Hamilton theorem.

The weaker property corresponding to stabilizability is **detectability**. A system is detectable if all unobservable modes are stable, so that although the state may not be reconstructed exactly, the estimation error can still be driven to zero asymptotically. Observability implies detectability,

but not every detectable system is observable. In practice, detectability is often the minimal property needed to construct a convergent observer or state estimator. Observability can be checked using the following condition

$$\text{if } \text{rank} \begin{pmatrix} \lambda_j I - A \\ C \end{pmatrix} = n \quad \forall \lambda_j \in \Lambda_A^+ \Rightarrow (A, C) \text{ is detectable}$$

where Λ_A^+ is the set of eigenvalues of A lying on or outside the unit circle.

Structural properties

Controllability and observability are exact structural properties of a model. Their relaxed counterparts, stabilizability and detectability, are often sufficient for controller and observer design and are especially relevant when only the unstable modes must be influenced or reconstructed.

2.3 Analysis of Nonlinear Discrete-Time Systems

The previous section dealt with discrete-time LTI systems, for which many results can be stated in terms of matrices and eigenvalues. For nonlinear systems, the situation is more subtle. Stability is no longer merely a property of the matrix A , but a **property of an equilibrium point** of the nonlinear dynamics. The appropriate conceptual framework is Lyapunov stability theory.

2.3.1 Lyapunov stability of nonlinear systems

Consider an autonomous nonlinear discrete-time system

$$x(k+1) = g(x(k)),$$

and let $\bar{x}$ be an equilibrium point, meaning that

$$g(\bar{x}) = \bar{x}.$$

Lyapunov stability formalizes the intuitive idea that if the system is perturbed only slightly from equilibrium, then it should remain close to equilibrium for all future time. More precisely, $\bar{x}$ is Lyapunov stable if for every $\varepsilon > 0$ there exists a $\delta(\varepsilon) > 0$ such that

$$\|x(0) - \bar{x}\| < \delta(\varepsilon) \implies \|x(k) - \bar{x}\| < \varepsilon \quad \forall k \geq 0.$$

If, in addition, trajectories converge to the equilibrium (i.e. is attractive), meaning that

$$\lim_{k \rightarrow \infty} \|x(k) - \bar{x}\| = 0 \quad \forall x(0) \in \mathbb{R}^n,$$

then the equilibrium is **globally asymptotically stable**.

2.3.2 Lyapunov functions

A standard way to prove stability is to construct a Lyapunov function. The intuition comes from mechanics: if the total energy of a damped system decreases over time, then the system tends to settle to an equilibrium. Lyapunov functions generalize this idea to arbitrary dynamical systems. For an equilibrium at the origin, a **Lyapunov function** is a scalar function

$$V : \mathbb{R}^n \rightarrow \mathbb{R}$$

that is continuous at the origin, finite everywhere, radially unbounded, positive definite, and strictly decreases along system trajectories. In particular, one requires

$$V(0) = 0, \quad V(x) > 0 \text{ for all } x \neq 0,$$

$$\|x\| \rightarrow \infty \implies V(x) \rightarrow \infty,$$

and

$$V(g(x)) - V(x) \leq -\alpha(x),$$

where α is a continuous positive definite function. **If such a function exists, then the origin is globally asymptotically stable.** If the decrease condition holds only with α positive semidefinite, then one obtains Lyapunov stability but not necessarily asymptotic convergence.

2.3.3 Indirect Lyapunov method

Although Lyapunov functions are powerful, they are often **difficult to construct** for general nonlinear systems. A useful alternative is the indirect Lyapunov method, which studies the linearization around the equilibrium. Let

$$A = \left. \frac{\partial g}{\partial x^\top} \right|_{x=0},$$

then the origin is **locally asymptotically stable** if all eigenvalues of A satisfy

$$|\lambda_i| < 1.$$

On the other hand, if at least one eigenvalue satisfies

$$|\lambda_i| > 1,$$

then the origin is unstable. The borderline case in which some eigenvalue lies exactly on the unit circle is **inconclusive**, and one must resort to a more refined analysis, often through a Lyapunov function.

This theorem is especially important because it connects nonlinear local stability with the familiar eigenvalue test from linear systems. It also clarifies why linearization is so useful: it can certify local asymptotic stability when the equilibrium lies safely inside the stable region, but it cannot always settle the question in marginal cases.

2.3.4 Lyapunov stability of linear systems

For **linear systems**, Lyapunov theory becomes constructive. Consider

$$x(k+1) = Ax(k).$$

A natural candidate Lyapunov function is quadratic,

$$V(x) = x^\top Px,$$

with $P = P^\top > 0$. Evaluating the change along trajectories gives

$$V(x(k+1)) - V(x(k)) = x(k)^\top (A^\top PA - P)x(k).$$

If one can choose $P > 0$ such that

$$A^\top PA - P = -Q, \quad Q > 0, \tag{2.1}$$

then

$$V(x(k+1)) - V(x(k)) = -x(k)^\top Q x(k) < 0$$

for all nonzero $x(k)$, which proves asymptotic stability. Equation (2.1) is the **discrete-time Lyapunov equation**.

A central theorem states that, for discrete-time LTI systems, the Lyapunov equation has a **unique positive definite solution** P for every given $Q > 0$ **if and only if** all eigenvalues of A lie inside the unit circle. Thus, in the linear case, eigenvalue stability and the existence of a quadratic Lyapunov function are equivalent. Moreover, because the system is linear, this stability is automatically global (stability is always "global" for linear systems, since the dynamics are the same everywhere in the state space).

There is also an important **interpretation** of the matrix P in terms of infinite-horizon cost. If the system is asymptotically stable and one defines

$$\Psi(x(0)) = \sum_{k=0}^{\infty} x(k)^\top Q x(k),$$

then this infinite sum can be written as

$$\Psi(x(0)) = x(0)^\top P x(0),$$

where P is precisely the solution of the discrete-time Lyapunov equation.

We can prove this by defining $H_k = (A^k)^\top Q (A^k)$ and $P = \sum_{k=0}^{\infty} H_k$. Notice that the limit of the sum exists because the system is asymptotically stable. Then we have that

$$A^\top H_k A = (A^{k+1})^\top Q (A^{k+1}) = H_{k+1}$$

Thus

$$A^\top P A = A^\top \left(\sum_{k=0}^{\infty} H_k \right) A = \sum_{k=0}^{\infty} A^\top H_k A = \sum_{k=0}^{\infty} H_{k+1} = P - H_0 = P - Q$$

In other words, the same quadratic function that certifies stability also represents the infinite-horizon cost-to-go associated with the stage cost $x^\top Q x$. This link is conceptually very important for MPC, because later the optimal cost itself will play the role of a Lyapunov function for the closed-loop system under suitable assumptions.

2.4 Summary

The material of this chapter provides the basic system-theoretic background required for MPC. Starting from nonlinear continuous-time models, one typically linearizes around an operating point and then discretizes the system to obtain a model suitable for digital control implementation. For linear discrete-time systems, explicit predictions over a finite horizon can be written as linear functions of the current state and the future input sequence, which is exactly the structure exploited in predictive control.

On the analysis side, stability of LTI systems is characterized by eigenvalue location, while controllability and observability determine whether the system can be influenced and reconstructed from measurements. Their weaker counterparts, stabilizability and detectability, are often sufficient for design tasks. For nonlinear systems, Lyapunov theory provides the appropriate framework to study stability of equilibria. In the linear case, quadratic Lyapunov functions arise naturally through the discrete-time Lyapunov equation and connect stability analysis with infinite-horizon cost. This bridge between dynamics, stability, and cost is one of the conceptual pillars on which the theory of MPC is built.

Chapter 3

Unconstrained Control

3.1 Introduction to Linear Quadratic Optimal Control

The next step toward model predictive control is to study optimal control problems in a setting where the model is **linear**, the objective is **quadratic**, and **no inequality constraints** are present. Although such problems are **much simpler** than full MPC formulations, they already contain many of the central ideas of predictive control: one optimizes over a finite or infinite horizon, one trades off state regulation against control effort through a cost function, and one exploits the model to predict future trajectories. In addition, this class of problems admits explicit structure and elegant solution methods, which makes it an ideal starting point.

In discrete-time optimal control, the goal is to choose an input sequence

$$U := [u_0^\top, u_1^\top, \dots]^\top$$

so as to minimize a performance criterion starting from a given initial state $x(0)$. In the finite-horizon case, a common objective is a sum of stage costs plus a terminal penalty,

$$J(x_0, U) := \ell(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i),$$

subject to the system dynamics

$$x_{i+1} = g(x_i, u_i), \quad i = 0, \dots, N-1,$$

the initial condition

$$x_0 = x(0),$$

and, in the most general setting, also input, state, and terminal constraints. In this chapter, however, we focus on the unconstrained linear quadratic case, which already reveals the main structure of optimal regulation and will later serve as a key building block in MPC.

3.1.1 Finite-horizon linear quadratic regulation

We consider the discrete-time **linear time-invariant** system

$$x(k+1) = Ax(k) + Bu(k),$$

and we seek to **regulate the state to the origin** by minimizing the quadratic cost

$$J^*(x(0)) := \min_U x_N^\top P x_N + \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i),$$

subject to

$$x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1,$$

and

$$x_0 = x(0).$$

Here $P \succeq 0$ is the terminal weight, $Q \succeq 0$ is the state weight, $R \succ 0$ is the input weight, and N is the horizon length. The matrices Q and R determine how strongly one penalizes state deviations and control effort, while the matrix P determines how the final state at the end of the horizon is valued. Because the problem is unconstrained and the objective is quadratic, one expects a solution with a particularly **simple structure**.

The interpretation of the cost is straightforward but important. The state term penalizes deviations from the origin, while the input term penalizes aggressive control action. The controller therefore balances performance against effort. A large Q relative to R encourages fast regulation at the price of stronger control inputs, whereas a large R makes the controller more conservative. This trade-off is one of the reasons why quadratic optimal control is both practical and conceptually appealing.

3.2 Batch Approach

A first way to solve the finite-horizon problem is to **eliminate the state variables** and express the entire optimization problem directly in terms of the initial state and the input sequence. This is called the batch approach. The key observation is that repeated substitution of the system dynamics allows all future states to be written explicitly as linear functions of $x(0)$ and the decision variables $u_0, \dots, u_{N-1}$.

Starting from $x_0 = x(0)$, one obtains

$$x_1 = Ax(0) + Bu_0,$$

$$x_2 = A^2x(0) + ABu_0 + Bu_1,$$

and, continuing in this way, all states up to x_N . Collecting them into the stacked vector

$$X := [x_0^\top, x_1^\top, \dots, x_N^\top]^\top,$$

and the inputs into

$$U := [u_0^\top, u_1^\top, \dots, u_{N-1}^\top]^\top,$$

one may write

$$X = S_x x(0) + S_u U,$$

where

$$S_x = \begin{bmatrix} I \\ A \\ A^2 \\ \vdots \\ A^N \end{bmatrix} \quad S_u = \begin{bmatrix} 0 & 0 & \dots & 0 \\ B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & \dots & AB & B \end{bmatrix}$$

To rewrite the cost compactly, define the block-diagonal matrices

$$\bar{Q} := \text{blockdiag}(Q, \dots, Q, P), \quad \bar{R} := \text{blockdiag}(R, \dots, R).$$

Then the cost becomes

$$J(x(0), U) = X^\top \bar{Q} X + U^\top \bar{R} U.$$

Substituting the expression for X yields

$$J(x(0), U) = (S_x x(0) + S_u U)^\top \bar{Q} (S_x x(0) + S_u U) + U^\top \bar{R} U.$$

After expansion, this can be written as

$$\begin{aligned} J(x(0), U) &= x(0)^\top S_x^\top \bar{Q} S_x x(0) + 2x(0)^\top \textcolor{red}{S}_x^\top \bar{Q} \textcolor{red}{S}_u U + U^\top \textcolor{blue}{S}_u^\top \bar{Q} \textcolor{blue}{S}_u U + U^\top \bar{R} U \\ &= U^\top \textcolor{blue}{H} U + 2x(0)^\top \textcolor{red}{F} U + x(0)^\top S_x^\top \bar{Q} S_x x(0) \end{aligned}$$

where

$$H := S_u^\top \bar{Q} S_u + \bar{R}, \quad F := S_x^\top \bar{Q} S_u.$$

Since $\bar{R} \succ 0$ and $S_u^\top \bar{Q} S_u \succeq 0$, it follows that $H \succ 0$. Therefore the cost is a strictly convex quadratic function of U , and hence admits a unique minimizer.

The optimal input sequence is obtained by differentiating with respect to U and setting the gradient to zero:

$$\nabla_U J(x(0), U) = 2HU + 2F^\top x(0) = 0.$$

Hence

$$U^*(x(0)) = -H^{-1}F^\top x(0).$$

So the optimal sequence is a linear function of the initial state. Substituting this optimizer back into the cost shows that the **optimal value is itself a quadratic function of the initial state**:

$$J^*(x(0)) = x(0)^\top \left(S_x^\top \bar{Q} S_x - S_x^\top \bar{Q} S_u (S_u^\top \bar{Q} S_u + \bar{R})^{-1} S_u^\top \bar{Q} S_x \right) x(0).$$

This is an important structural result: in linear quadratic optimal control, not only the optimal input, but also the optimal cost-to-go, has a quadratic dependence on the state.

Batch solution

In the unconstrained finite-horizon LQ problem, all predicted states can be eliminated in favor of the input sequence, and the resulting optimization becomes a strictly convex quadratic minimization in the stacked input vector U . Its unique solution is linear in the initial state.

The batch method is conceptually direct and easy to derive. However, it returns a numerical sequence of future inputs tailored to the specific initial condition. It does not directly provide a state-feedback law to be used later if the trajectory deviates from the nominal prediction. This limitation motivates a second viewpoint, based on dynamic programming. Plus, if there are state or input constraints, solving this problem by matrix inversion is not guaranteed to result in a feasible solution.

3.3 Recursive Approach and Dynamic Programming

Instead of solving for the entire input sequence at once, one may exploit the recursive structure of the problem. The central idea is to describe the optimal cost from a given time onward as a function of the current state. This leads naturally to dynamic programming and to Bellman's principle of optimality.

3.3.1 Cost-to-go and one-step optimization

For a time index j , define the j -step **optimal cost-to-go** as

$$J_j^*(x(j)) := \min_{U_{j \rightarrow N}} x_N^\top P x_N + \sum_{i=j}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i),$$

subject to the dynamics from time j onward and the initial condition $x_j = x(j)$. This is the minimum cost achievable over the remaining portion of the horizon starting from the state at

time j .

At the last stage, namely $j = N - 1$, the problem reduces to a one-step optimization:

$$J_{N-1}^*(x_{N-1}) = \min_{u_{N-1}} \left(x_{N-1}^\top Q x_{N-1} + u_{N-1}^\top R u_{N-1} + x_N^\top P_N x_N \right),$$

with

$$x_N = A x_{N-1} + B u_{N-1}, \quad P_N = P.$$

Substituting the dynamics into the cost yields a quadratic expression in u_{N-1} :

$$\begin{aligned} J_{N-1}^*(x_{N-1}) &= \min_{u_{N-1}} \left(x_{N-1}^\top Q x_{N-1} + u_{N-1}^\top R u_{N-1} + (A x_{N-1} + B u_{N-1})^\top P_N (A x_{N-1} + B u_{N-1}) \right) \\ &= \min_{u_{N-1}} \left(x_{N-1}^\top (Q + A^\top P_N A) x_{N-1} + u_{N-1}^\top (R + B^\top P_N B) u_{N-1} + 2 x_{N-1}^\top A^\top P_N B u_{N-1} \right) \end{aligned}$$

Setting its gradient to zero leads to the optimal terminal-step control law

$$2(B^\top P_N B + R)u_{N-1} + 2B^\top P_N A x_{N-1} = 0$$

$$\begin{aligned} u_{N-1}^* &= -(B^\top P_N B + R)^{-1} B^\top P_N A x_{N-1} \\ &=: F_{N-1} x_{N-1}. \end{aligned}$$

The corresponding optimal cost-to-go turns out to be quadratic:

$$J_{N-1}^*(x_{N-1}) = x_{N-1}^\top P_{N-1} x_{N-1},$$

where

$$P_{N-1} = A^\top P_N A + Q - A^\top P_N B (B^\top P_N B + R)^{-1} B^\top P_N A.$$

This already reveals the characteristic structure of the recursive solution: the **optimal control law is linear in the state, and the optimal cost-to-go is quadratic**.

3.3.2 Bellman's principle of optimality

Now consider the problem one step earlier. The optimal two-step cost from time $N - 2$ can be written as

$$J_{N-2}^*(x_{N-2}) = \min_{u_{N-2}} \left(x_{N-2}^\top Q x_{N-2} + u_{N-2}^\top R u_{N-2} + J_{N-1}^*(x_{N-1}) \right),$$

subject to

$$x_{N-1} = A x_{N-2} + B u_{N-2}.$$

The reason this is valid is **Bellman's principle of optimality**: any tail of an optimal trajectory must itself be optimal for the subproblem starting at that later point. In other words, if a policy is optimal over the full horizon, then after any intermediate time the remaining decisions must solve the corresponding shorter-horizon optimal control problem.

This principle gives the recursion

$$J_i^*(x_i) = \min_{u_i} \left(x_i^\top Q x_i + u_i^\top R u_i + J_{i+1}^*(x_{i+1}) \right), \quad x_{i+1} = A x_i + B u_i,$$

for all $i = 0, \dots, N - 1$. **Assuming** that

$$J_{i+1}^*(x_{i+1}) = x_{i+1}^\top P_{i+1} x_{i+1},$$

one again obtains by substitution and differentiation the optimal policy

$$u_i^* = -(B^\top P_{i+1} B + R)^{-1} B^\top P_{i+1} A x_i =: F_i x_i,$$

and the recursion

$$P_i = A^\top P_{i+1} A + Q - A^\top P_{i+1} B (B^\top P_{i+1} B + R)^{-1} B^\top P_{i+1} A.$$

This backward recursion, initialized with $P_N = P$, is called the **discrete-time Riccati equation**, or Riccati difference equation. It yields all feedback gains F_i and all cost matrices P_i over the horizon. In particular,

$$J^*(x(0)) = J_0^*(x(0)) = x(0)^\top P_0 x(0).$$

Thus the recursive method computes a **family of feedback laws, one for each stage**, rather than only a single open-loop sequence.

Riccati recursion

For the finite-horizon unconstrained LQ problem, the optimal cost-to-go has the quadratic form

$$J_i^*(x_i) = x_i^\top P_i x_i,$$

and the optimal control law has the linear state-feedback form

$$u_i^* = F_i x_i,$$

where the matrices P_i are computed backward through the Riccati difference equation.

3.3.3 Comparison of batch and recursive viewpoints

The batch and recursive approaches solve the same unconstrained problem, but they do so from different perspectives. The batch method returns the optimal input sequence as a stacked vector of numerical values depending on the initial state. The recursive method returns feedback laws depending on the state at each stage. If the system evolves exactly as predicted, then both methods generate the same control actions along the nominal trajectory.

The recursive viewpoint has two major conceptual advantages. First, it exposes the feedback structure of the solution. If disturbances or model mismatch cause the actual state to deviate from the nominal prediction, the control decision can still be recomputed optimally from the current state using the feedback law. Second, it is computationally more attractive in many settings, because it breaks the large horizon problem into a backward sequence of smaller problems. The batch Hessian grows with the horizon length, while the recursive method exploits the problem structure much more efficiently. On the other hand, once constraints are added, the batch formulation is usually much easier to generalize, which is one reason why MPC is commonly formulated as a finite-horizon constrained optimization over a stacked input sequence.

3.4 Receding Horizon Interpretation

The unconstrained finite-horizon LQ problem also provides a simple instance of receding horizon control. One solves the finite-horizon optimization problem, extracts the first control input from the optimal sequence, applies it to the system, and then repeats the entire procedure at the next sampling instant. This is exactly the same logic that later defines MPC in the constrained case.

For an unconstrained linear quadratic problem, applying this receding-horizon strategy yields a **linear state-feedback controller**. Still, the interpretation is important: the **controller is**

not derived first and optimized later; instead, it emerges from the repeated solution of a finite-horizon optimal control problem. This viewpoint generalizes naturally to more complex models and to constrained problems, where no closed-form feedback law is available.

3.5 Infinite-Horizon LQR

In many situations, the natural control objective is not limited to a finite planning window, but extends indefinitely into the future. This leads to the infinite-horizon linear quadratic regulator problem

$$J_\infty(x(0)) = \min_{u(\cdot)} \sum_{i=0}^{\infty} (x_i^\top Q x_i + u_i^\top R u_i),$$

subject to

$$x_{i+1} = Ax_i + Bu_i, \quad i = 0, 1, 2, \dots,$$

and

$$x_0 = x(0).$$

The infinite horizon removes the arbitrariness of a chosen terminal time and leads to a **time-invariant optimal feedback law**.

3.5.1 Algebraic Riccati equation

By analogy with the finite-horizon recursive solution, one expects the infinite-horizon optimal cost-to-go to have the quadratic form

$$J_\infty^*(x) = x^\top P_\infty x,$$

and the corresponding optimal control law to be

$$u^*(k) = -(B^\top P_\infty B + R)^{-1} B^\top P_\infty A x(k) =: F_\infty x(k).$$

If the finite-horizon Riccati difference equation converges as the horizon tends to infinity, then the limiting matrix P_∞ must satisfy the fixed-point equation

$$P_\infty = A^\top P_\infty A + Q - A^\top P_\infty B (B^\top P_\infty B + R)^{-1} B^\top P_\infty A.$$

This is the **discrete-time algebraic Riccati equation**. Its solution yields the asymptotic LQR feedback gain F_∞ .

Under standard assumptions, namely stabilizability of (A, B) and observability of $(Q^{1/2}, A)$, the Riccati recursion converges to the unique positive definite solution P_∞ of the algebraic Riccati equation. This gives a well-defined infinite-horizon optimal controller. The corresponding **feedback is time-invariant**, in contrast with the finite-horizon case where the gains F_i vary with the remaining length of the horizon.

3.5.2 Stability of infinite-horizon LQR

An important advantage of the infinite-horizon LQR is that, under suitable assumptions, the resulting closed-loop system is asymptotically stable. More precisely, if (A, B) is stabilizable and $Q, R \succ 0$, then the optimal infinite-horizon feedback law

$$u(k) = F_\infty x(k), \quad F_\infty = -(R + B^\top P_\infty B)^{-1} B^\top P_\infty A,$$

renders the closed-loop system

$$x(k+1) = (A + BF_\infty)x(k)$$

asymptotically stable. The proof is based on the observation that the optimal value function of the infinite-horizon problem is a Lyapunov function for the closed-loop system.

Lyapunov function for infinite-horizon LQR

If (A, B) is stabilizable and $Q, R \succ 0$, then the optimal value function

$$J^*(x) = x^\top P_\infty x$$

is a Lyapunov function for the closed-loop system

$$x^+ = (A + BF_\infty)x,$$

where

$$F_\infty = -(R + B^\top P_\infty B)^{-1} B^\top P_\infty A,$$

and $P_\infty \succ 0$ is the solution of the algebraic Riccati equation.

The first two Lyapunov properties follow immediately from $P_\infty \succ 0$. Indeed,

$$J^*(0) = 0, \quad J^*(x) = x^\top P_\infty x > 0 \quad \forall x \neq 0,$$

and, since P_∞ is positive definite,

$$\|x\| \rightarrow \infty \implies J^*(x) = x^\top P_\infty x \rightarrow \infty.$$

It therefore remains to show that J^* decreases strictly along closed-loop trajectories.

To do so, observe that the optimal infinite-horizon cost starting at time k can be written as

$$J^*(x(k)) = x(k)^\top P_\infty x(k) = \sum_{i=k}^{\infty} x(i)^\top (Q + F_\infty^\top R F_\infty) x(i),$$

because along the optimal closed-loop trajectory one has

$$u(i) = F_\infty x(i).$$

Now consider the value of the same function one step later:

$$J^*(x(k+1)) = J^*((A + BF_\infty)x(k)) = \sum_{i=k+1}^{\infty} x(i)^\top (Q + F_\infty^\top R F_\infty) x(i).$$

Comparing the two expressions gives

$$J^*(x(k+1)) = J^*(x(k)) - x(k)^\top (Q + F_\infty^\top R F_\infty) x(k).$$

Since $Q \succ 0$ and $R \succ 0$, the matrix

$$Q + F_\infty^\top R F_\infty$$

is positive definite, and therefore

$$x(k)^\top (Q + F_\infty^\top R F_\infty) x(k) > 0 \quad \forall x(k) \neq 0.$$

Hence

$$J^*(x(k+1)) < J^*(x(k)) \quad \forall x(k) \neq 0.$$

Thus the optimal value function decreases strictly along every nontrivial closed-loop trajectory, and is therefore a Lyapunov function for the system

$$x(k+1) = (A + BF_\infty)x(k).$$

This proves that the origin is asymptotically stable. In particular,

$$\lim_{k \rightarrow \infty} x(k) = 0.$$

The result is conceptually important: the same function that measures the optimal infinite-horizon performance also certifies stability of the optimal closed loop.

The argument above used the stronger assumption $Q \succ 0$. This condition can be relaxed. In particular, the infinite-horizon LQR remains stabilizing under the weaker assumptions

$$Q \succeq 0, \quad R \succ 0,$$

provided that (A, B) is stabilizable and $(Q^{1/2}, A)$ is detectable. Thus detectability plays here the same role that observability played in the stronger result: it is sufficient to ensure that no unstable mode can remain invisible to the cost.

3.6 Terminal Weight and Quasi-Infinite-Horizon Thinking

Since MPC is usually implemented with a finite prediction horizon, one naturally asks how to choose the terminal weight P so that the finite-horizon problem approximates the infinite-horizon behavior as well as possible. There are several important possibilities.

A particularly natural choice is to set the terminal weight equal to the infinite-horizon cost-to-go beyond the horizon. In other words, one chooses

$$P = P_\infty,$$

where P_∞ solves the algebraic Riccati equation. Then the finite-horizon problem exactly matches the infinite-horizon solution from the terminal point onward. This is the cleanest way of embedding the tail of the infinite-horizon problem into a finite-horizon formulation.

A second possibility is to assume that no control action is applied after the end of the horizon, so that for $k \geq N$ the state evolves autonomously according to

$$x(k+1) = Ax(k).$$

In that case, the appropriate terminal weight is obtained from the Lyapunov equation

$$A^\top P A + Q = P.$$

This choice only makes sense if the autonomous system is asymptotically stable, because otherwise no positive definite solution exists.

A third option is to impose that both the state and the input vanish after the horizon. In this case one does not only use a terminal penalty, but rather imposes a terminal constraint such as

$$x_N = 0.$$

This viewpoint is important for later constrained MPC theory, where terminal costs and terminal sets are used precisely to mimic infinite-horizon behavior and recover stability guarantees from a finite optimization horizon.

3.7 Summary

Unconstrained linear quadratic optimal control is the simplest predictive-control setting in which all central ingredients of MPC already appear. One starts from a model, predicts future trajectories over a horizon, and chooses the control sequence that minimizes a quadratic objective balancing state regulation and control effort. In the finite-horizon case, the problem can be solved either in batch form, by eliminating the states and minimizing a quadratic function of the full input sequence, or recursively, through dynamic programming and the Riccati difference equation.

The recursive viewpoint is especially important because it reveals the feedback structure of the solution and leads directly to Bellman's principle of optimality. It also clarifies the meaning of receding-horizon control: at each step, one solves a finite-horizon problem, applies the first input, and repeats the optimization at the next sampling instant. The quality of this strategy depends strongly on the horizon length and on the terminal ingredients used.

Passing to the infinite-horizon problem yields the time-invariant LQR feedback law, determined by the algebraic Riccati equation. Under standard stabilizability and detectability assumptions, the infinite-horizon optimal value function is quadratic and also serves as a Lyapunov function, thereby proving stability of the optimal closed loop. This connection between optimization, value functions, and Lyapunov stability will be one of the central themes throughout the rest of MPC theory.

Chapter 4

Introduction to Convex Optimization

Part II

Nominal MPC

Chapter 5

Constrained Finite Time Optimal Control

Chapter 6

Invariance

Chapter 7

Feasibility and Stability

Part III

Practical MPC

Chapter 8

Practical Issues in MPC

Part IV

Robust MPC

Chapter 9

Robust MPC 1

Chapter 10

Robust MPC 2

Part V

Implementation Methods

Chapter 11

Implementation Methods